Q3.2

To split the dataset into the desired training ratios, I used sklearn's train_test_split() function and trained a 1-nearest neighbour model using both my implementation and sklearn's.

```
def question_3_2(X, Y):
    # The splits required by the assignment
    split_configs = {
        "80/10/10": (0.80, 0.10, 0.10),
        "34/33/33": (0.34, 0.33, 0.33),
        "25/25/50": (0.25, 0.25, 0.50)
    }
    metrics = ["Euclidean", "Cosine"]
    k = 1 # 1 nearest neighbour for this section

    for split_name, (train_r, val_r, test_r) in split_configs.items():
        print(f"\n{'='*30}")
        print(f"=== Data Split: {split_name} ===")
        print(f"{'='*30}")

        # Split off the test set first
        X_temp, X_test, Y_temp, Y_test = train_test_split(
            X, Y, test_size=test_r, shuffle=False
        )

        # Split the remaining data into train and validation
        val_ratio_adjusted = val_r / (train_r + val_r)
        X_train, X_val, Y_train, Y_val = train_test_split(
            X_temp, Y_temp, test_size=val_ratio_adjusted, shuffle=False
        )

        for metric in metrics:
            run_evaluation(X_train, Y_train, X_val, Y_val, X_test, Y_test, k,
metric)

            # predict and find accuracy using SkLearn
            sk_knn = KNeighborsClassifier(n_neighbors=k, metric=metric.lower())
            sk_knn.fit(X_train, Y_train)

            sk_train_acc = accuracy_score(Y_train, sk_knn.predict(X_train))
            sk_train_err = 1 - sk_train_acc
            sk_val_acc = accuracy_score(Y_val, sk_knn.predict(X_val))
            sk_val_err = 1 - sk_val_acc
            sk_test_acc = accuracy_score(Y_test, sk_knn.predict(X_test))
            sk_test_err = 1 - sk_test_acc
            print(f"SkLearn Metrics for k={k}, distance={metric}:")
```

```
            print(f"Training Accuracy: {sk_train_acc:.4f} (Error:
{sk_train_err:.4f})")
            print(f"Validation Accuracy: {sk_val_acc:.4f} (Error:
{sk_val_err:.4f})")
            print(f"Test Accuracy: {sk_test_acc:.4f} (Error:
{sk_test_err:.4f})\n")
```

The output from this code follows in Table 2:

Table 2: Data Splits and Validation Error Rate Across Different Metrics

Split	Metric	Custom Validation Error Rate (%)	Sklearn Validation Error Rate (%)
80/10/10	Euclidean	36.36	36.36
80/10/10	Cosine	42.86	42.86
34/33/33	Euclidean	35.43	35.43
34/33/33	Cosine	43.31	43.31
25/25/50	Euclidean	35.94	35.94
25/25/50	Cosine	49.48	49.48

Observing these validation error rates across the different splits and metrics for both our custom and sklearn implementation, the 34/33/33 split with Euclidean distance has the lowest validation error rate. Across all splits, Euclidean distance gives lower validation error rates. This makes logical sense given the features, as Glucose, BMI and Age should be related through numerical distance. For example, a patient with Glucose of 100 and BMI of 25 would have a similar vector angle to someone with Glucose of 200 and BMI of 50, but these are two very different health outcomes. In typical machine learning examples, an 80/10/10 split usually performs best but because we are dealing with a relatively small dataset, the 33% validation set happened to map very cleanly to the structure of its 34% training set.